Your first milestone: Learn sorting and selection, interval scheduling, and running-maximum greedy until you can recognize them from the problem statement alone. These will give you a strong foundation before tackling heaps, advanced scheduling, and graph algorithms.

You said:

Create an Infosys SP/DSE greedy algorithm learning plan. Language: Java. Current DSA level: intermediate. Preferred learning style: recognition. Teach pattern recognition, examples, correctness intuition, complexity, and practice problems in a progressive order.

ChatGPT said:

Infosys SP/DSE Greedy Algorithms — Java Roadmap

Since you're at an intermediate DSA level, the goal is not to memorize dozens of greedy solutions. It's to learn to look at a new problem and recognize **which greedy pattern fits, what decision to make, and why that decision is safe**.

We'll use Java throughout, progressing from sorting-based greedy algorithms to heaps, scheduling, and graph algorithms.

1. Your learning roadmap

Phase 1: Sort and select

Recognize maximum-count and selection problems. Learn sorting, feasibility checks, and two pointers.

Practice: Assign Cookies, Boats to Save People.

Phase 2: Intervals and deadlines

Identify start/end times, overlapping intervals, and scheduling constraints.

Practice: Activity Selection, Non-overlapping Intervals, Job Sequencing.

Phase 3: Ratios and resource allocation

Learn value-per-unit decisions and constrained selection.

Practice: Fractional Knapsack, Maximum Units on a Truck.

Phase 4: Running greedy and feasibility

Maintain a running maximum, balance, or reachable boundary without sorting.

Practice: Jump Game, Jump Game II, Gas Station.

Phase 5: Priority queues and replacement

Repeatedly choose the best candidate, or undo a previous selection when a better choice appears.

Practice: Minimum Cost of Ropes, Reorganize String, Course Schedule III.

Phase 6: Advanced greedy

Apply greedy choices to graph edges, shortest paths, and character frequencies.

Practice: Kruskal's MST, Dijkstra, Reorganize String.

This is a structured preparation roadmap, not a claim about the exact distribution of questions in the Infosys SP/DSE exam.

2. The recognition decision tree

Before coding, classify the problem by its objective and structure.

What are you optimizing?

Maximum count

Select as many compatible items as possible.

→ Sort and select; interval scheduling

Minimum cost

Combine items or allocate resources cheaply.

→ Min-heap, sometimes graph greedy

Maximum profit

Items have value, weight, or deadlines.

→ Ratio, profit sorting, or heap replacement

Reachability

Can you reach a destination or maintain feasibility?

→ Running maximum or balance

Then ask yourself:

- Are there start and end times? Consider interval scheduling.
- Are there values and weights? Consider ratios, but check whether fractions are allowed.
- Does the best candidate change after every operation? Consider a priority queue.

- Can you summarize everything important with a running maximum or minimum? Consider a linear greedy scan.
- Can an earlier choice need to be reconsidered? Consider heap-based greedy, dynamic programming, or another approach.

These clues help you choose a candidate algorithm. You still need to verify that its decisions lead to an optimal answer.

3. Six core greedy patterns in Java

Pattern 1 — Sort and select

Recognition clue: The problem asks for the maximum number of satisfied people, selected items, or feasible pairs.

Example: Assign Cookies. Each child has a greed factor, and each cookie has a size.

java

```
import java.util.Arrays;

class Solution {
    public int findContentChildren(int[] g, int[] s) {
        Arrays.sort(g);
        Arrays.sort(s);

        int i = 0, j = 0;

        while (i < g.length && j < s.length) {
            if (s[j] >= g[i]) {
                i++;
            }
            j++;
        }

        return i;
    }
}
```

Why it works: Satisfy the least demanding remaining child using the smallest cookie that satisfies them. This preserves larger cookies for children who need them.

- Time: $O(n \log n + m \log m)$
- Extra space: $O(1)$ auxiliary space, ignoring sorting implementation details.

Practice in order:

1. Assign Cookies ↗
2. Boats to Save People ↗
3. Bag of Tokens ↗

Pattern 2 — Interval scheduling

Recognition clue: The input contains start/end times, meetings, events, or overlapping intervals.

For selecting the maximum number of non-overlapping activities, sort by **end time ascending**.

java

```
import java.util.Arrays;

class Solution {
    public int maxActivities(int[][] activities) {
        Arrays.sort(activities,
            (a, b) -> Integer.compare(a[1], b[1]));

        int count = 0;
        int lastEnd = Integer.MIN_VALUE;

        for (int[] activity : activities) {
            int start = activity[0];
            int end = activity[1];

            if (start >= lastEnd) {
                count++;
                lastEnd = end;
            }
        }
```

```
        return count;
    }
}
```

Why it works: Choosing the compatible activity that finishes earliest leaves the greatest opportunity for future activities. Repeatedly making this choice produces an optimal schedule.

- Time: $O(n \log n)$
- Auxiliary space: $O(1)$ beyond sorting overhead.

The code allows one activity to start exactly when another ends.

Important distinction:

Objective	Typical sorting rule
Select maximum non-overlapping activities	End time ascending
Merge overlapping intervals	Start time ascending
Find minimum removals to eliminate overlaps	End time ascending
Schedule jobs with profits and deadlines	Profit descending, then find a feasible slot

Practice:

- Maximum Meetings in One Room ↗
- Non-overlapping Intervals ↗
- Merge Intervals ↗

Pattern 3 — Sort by ratio

Recognition clue: Each item has a value and a weight or resource cost, and fractions of items are allowed.

For fractional knapsack, sort by value per unit weight.

java

```
import java.util.Arrays;

class Solution {
    public double fractionalKnapsack(
            int capacity, int[][] items) {
```

```
        // Each item: {value, weight}
        Arrays.sort(items, (a, b) ->
            Double.compare(
                (double) b[0] / b[1],
                (double) a[0] / a[1]
            )
        );

        double total = 0.0;

        for (int[] item : items) {
            int value = item[0];
            int weight = item[1];

            int take = Math.min(capacity, weight);
            total += (double) take * value / weight;
            capacity -= take;

            if (capacity == 0) {
                break;
            }
        }

        return total;
    }
}
```

Why it works: Each unit of capacity should go to an available item with the highest value per unit weight.

- Time: $O(n \log n)$
- Extra space: depends on the sorting implementation.

Trap: This ratio rule is not generally optimal for 0/1 knapsack, where items cannot be divided.

Practice:

- Fractional Knapsack ↗
- Maximum Units on a Truck ↗

Pattern 4 — Running maximum

Recognition clue: Each array element determines how far you can go, and you need to determine reachability.

Example: Jump Game.

java

```
class Solution {
    public boolean canJump(int[] nums) {
        int farthest = 0;

        for (int i = 0; i < nums.length; i++) {
            if (i > farthest) {
                return false;
            }

            farthest = Math.max(farthest, i + nums[i]);

            if (farthest >= nums.length - 1) {
                return true;
            }
        }

        return true;
    }
}
```

Why it works: `farthest` tracks the maximum reachable index from all positions processed so far. If the current index exceeds it, that index cannot be reached.

- Time: $O(n)$
- Extra space: $O(1)$

Practice:

- Jump Game ↗
- Jump Game II ↗
- Gas Station ↗

Pattern 5 — Repeatedly choose the smallest item using a min-heap

Recognition clue: You repeatedly combine items and want to minimize the total cost.

Example: Connect ropes with minimum cost.

java

```
import java.util.PriorityQueue;

class Solution {
    public long minCost(long[] ropes) {
        PriorityQueue<Long> pq = new PriorityQueue<>();

        for (long rope : ropes) {
            pq.offer(rope);
        }

        long cost = 0;

        while (pq.size() > 1) {
            long a = pq.poll();
            long b = pq.poll();

            long merged = a + b;
            cost += merged;
            pq.offer(merged);
        }

        return cost;
    }
}
```

Why it works: Repeatedly combining the two shortest ropes is optimal for the minimum-cost merge process.

- Time: $O(n \log n)$
- Space: $O(n)$

Practice:

- Minimum Cost of Ropes ↗

- Reorganize String ↗
- Course Schedule III ↗

Pattern 6 — Profit and deadlines

Recognition clue: Each job has a profit and deadline, and each job takes exactly one time slot.

The standard approach is to sort by profit descending, then schedule each job in the latest available slot at or before its deadline.

java

```
import java.util.Arrays;

class Solution {
    public int[] jobSequencing(int[][] jobs) {
        // Each job: {deadline, profit}
        Arrays.sort(jobs, (a, b) ->
            Integer.compare(b[1], a[1]));

        int maxDeadline = 0;
        for (int[] job : jobs) {
            maxDeadline = Math.max(maxDeadline, job[0]);
        }

        boolean[] slots = new boolean[maxDeadline + 1];

        int count = 0;
        int profit = 0;

        for (int[] job : jobs) {
            int deadline = Math.min(job[0], maxDeadline);

            for (int t = deadline; t >= 1; t--) {
                if (!slots[t]) {
                    slots[t] = true;
                    count++;
                    profit += job[1];
                    break;
                }
            }
```

```
        }

        return new int[]{count, profit};
    }
}
```

Why it works: Prioritizing profitable jobs preserves their value, and placing each selected job as late as possible leaves earlier slots available for jobs with tighter deadlines.

- Time: $O(n \log n + nD)$, where D is the maximum deadline.
- Space: $O(D)$.

This strategy applies to the unit-duration job-sequencing problem, not every scheduling problem.

Practice:

- Job Sequencing Problem ↗
- Course Schedule III ↗

4. Three-week preparation schedule

Aim for 60–90 minutes daily.

Week 1 — Sorting-based greedy

- Days 1–2: Assign Cookies, Boats to Save People.
- Days 3–4: Activity Selection, Non-overlapping Intervals.
- Day 5: Fractional Knapsack.
- Days 6–7: Mixed problems without looking at solutions.

Week 2 — Running greedy and heaps

- Days 8–9: Jump Game, Jump Game II.
- Days 10–11: Job Sequencing.
- Days 12–13: Minimum Cost of Ropes, Reorganize String.
- Day 14: Mixed practice and revision.

Week 3 — Advanced reasoning

- Days 15–16: Course Schedule III.

- Days 17–18: Kruskal's MST and Dijkstra.
- Days 19–20: Greedy versus dynamic programming.
- Day 21: Timed mock practice and review.

5. Learn to prove your greedy choice

For every problem, answer these five questions:

1. **Objective:** What exactly am I maximizing or minimizing?
2. **Choice:** What is the locally best candidate?
3. **Feasibility:** When can I accept or reject it?
4. **Correctness:** Why does this choice not make the final answer worse?
5. **Complexity:** Can I explain the time and space complexity?

A useful counterexample is the coin system `[1, 3, 4]` with target `6`. Choosing the largest available coin first gives `4 + 1 + 1`, using three coins, while `3 + 3` uses only two. This teaches you not to assume greedy is correct simply because the problem asks for a minimum.

6. Start your first recognition drill

Problem: You have several meetings with start and end times. You can attend only one meeting at a time, but you may attend one meeting that starts exactly when the previous one ends. Your goal is to attend the maximum number of meetings.

Which strategy should you try first?

- Sort by start time and select the earliest starting meeting.
- Sort by end time and select the earliest finishing compatible meeting.
- Sort by duration and always select the shortest meeting.
- Keep the original input order.

Check answer

Your first milestone: Become fluent in sort-and-select, interval scheduling, and running-maximum greedy. Once you can identify those patterns from the problem statement and justify them, move on to heaps and advanced scheduling.

This might also interest you.



HackerRank Ad

Put Your AI Skills to the Test

Ready for a coding challenge? Join HackerRank's Orchestrate hackathon